

Streamlit + FastAPI Chatbot

10번 강의와 달라지는 점

10번:

```
Streamlit -> Groq
```

11번:

```
Streamlit -> FastAPI -> Groq
```

Streamlit은 더 이상 Groq SDK를 직접 알 필요가 없습니다.
화면은 HTTP API만 호출합니다.

왜 FastAPI를 사이에 둘까?

Streamlit만으로도 챗봇은 만들 수 있습니다.

하지만 서비스 구조로 보면 FastAPI 백엔드를 두는 편이 더 좋습니다.

- API Key를 화면 코드에서 분리할 수 있습니다.
- 모바일 앱, 웹 프론트엔드, 관리자 페이지가 같은 API를 사용할 수 있습니다.
- Swagger UI로 요청/응답을 먼저 테스트할 수 있습니다.
- 검증, 오류 처리, 스트리밍 응답을 백엔드에서 통제할 수 있습니다.

전체 구조

```
사용자
|
v
Streamlit frontend/app.py
|
| HTTP POST /chat/stream
v
FastAPI backend/main.py
|
| GROQ_API_KEY
v
Groq LLM
```

프로젝트 구조

Chatbot with FastAPI/

└─ backend/	
└─ main.py	<- FastAPI 앱 생성 + 라우터 등록
└─ core/	
└─ config.py	<- 환경변수, 앱 메타데이터
└─ groq_client.py	<- Groq 클라이언트 생성
└─ schemas/	
└─ chat.py	<- 요청/응답 Pydantic 모델
└─ services/	
└─ chat_service.py	<- Groq 호출, SSE 데이터 생성
└─ routers/	
└─ health.py	<- /, /health
└─ chat.py	<- /chat, /chat/stream
└─ frontend/	
└─ app.py	<- Streamlit 화면 조립
└─ common/	
└─ config.py	<- 프론트 환경변수와 기본값
└─ message.py	<- 대화 메시지 생성
└─ api_client.py	<- FastAPI 호출, SSE 파싱
└─ .env.example	<- 환경변수 예시

backend 모듈화 기준

계층	담당
backend/main.py	앱 생성과 라우터 연결
backend/routers/	URL과 HTTP 응답 방식
backend/schemas/	요청/응답 데이터 모양
services/	실제 Groq 호출과 스트리밍 처리
backend/core/	환경변수와 외부 클라이언트

frontend 모듈화 기준

파일	담당
frontend/app.py	Streamlit UI와 화면 흐름
frontend/common/config.py	FastAPI URL, 기본 옵션
frontend/common/message.py	role , content 메시지 생성
frontend/common/api_client.py	/health , /chat/stream 호출

FastAPI(backend) 엔드포인트

메서드	경로	기능
GET	/	서버 안내
GET	/health	서버와 API Key 설정 상태 확인
POST	/chat	일반 JSON 챗봇 응답
POST	/chat/stream	SSE 스트리밍 챗봇 응답
GET	/docs	Swagger UI

요청 데이터

POST /chat , POST /chat/stream 은 같은 요청 모양을 사용합니다.

```
{  
  "message": "Streamlit과 FastAPI를 분리하면 어떤 장점이 있어?",  
  "history": [],  
  "temperature": 0.7,  
  "max_tokens": 800  
}
```

요청 검증

FastAPI는 Pydantic 모델로 요청을 검증합니다.

필드	조건
message	1자 이상, 2000자 이하
history	최대 20개 메시지
temperature	0.0 이상, 2.0 이하
max_tokens	1 이상, 4096 이하

조건을 어기면 FastAPI가 자동으로 422 응답을 반환합니다.

FastAPI 서버 코드 핵심

```
def create_app() -> FastAPI:
    app = FastAPI(
        title=APP_TITLE,
        description=APP_DESCRIPTION,
        version=APP_VERSION,
    )

    app.include_router(health_router)
    app.include_router(chat_router)

    return app
```

FastAPI 서버는 화면을 만들지 않습니다.

`main.py` 는 앱을 만들고 라우터만 연결합니다.

Groq 클라이언트 생성

backend/core/groq_client.py

```
def get_client() -> Groq:
    global _client

    if _client is None:
        _client = Groq(api_key=require_groq_api_key())

    return _client
```

API Key는 `.env`에만 둡니다.

Streamlit 화면 코드에는 Groq API Key를 넣지 않습니다.

일반 JSON 응답

backend/services/chat_service.py

```
response = get_client_for_request().chat.completions.create(  
    model=GROQ_MODEL,  
    messages=build_messages(req),  
    temperature=req.temperature,  
    max_completion_tokens=req.max_tokens,  
)
```

POST /chat 은 답변이 완성될 때까지 기다렸다가 JSON으로 반환합니다.

스트리밍 응답

backend/services/chat_service.py

```
stream = client.chat.completions.create(  
    model=GROQ_MODEL,  
    messages=build_messages(req),  
    temperature=req.temperature,  
    max_completion_tokens=req.max_tokens,  
    stream=True,  
)
```

POST /chat/stream 은 답변 조각을 받는 즉시 Streamlit으로 전달합니다.

Server-Sent Events(SSE) 형식

- SSE 는 웹 서버가 클라이언트(브라우저)에게 실시간으로 데이터를 계속 푸시(push) 할 수 있게 해주는 기술입니다.
- FastAPI 는 스트리밍 데이터를 Server-Sent Events 형식으로 보냅니다.

```
data: {"token": "안녕"}
```

```
data: {"token": "하세요"}
```

```
data: [DONE]
```

Streamlit은 이 줄을 읽고 token 만 꺼내 화면에 이어 붙입니다.

Streamlit(frontend) 화면 코드 핵심

```
if "messages" not in st.session_state:  
    st.session_state.messages = []
```

Streamlit은 새로고침처럼 스크립트가 다시 실행되는 구조입니다.
대화 기록은 `st.session_state` 에 저장해야 화면에 계속 남습니다.

Streamlit(frontend)에서 FastAPI(backend) 호출

frontend/common/api_client.py

```
with requests.post(
    f"{api_base_url}/chat/stream",
    json=payload,
    stream=True,
    timeout=(5, 120),
) as response:
    response.raise_for_status()
```

Streamlit은 Groq를 직접 호출하지 않습니다.
FastAPI의 `/chat/stream` API만 호출합니다.

화면에 스트리밍 출력

frontend/app.py

```
answer = st.write_stream(  
    stream_answer(  
        api_base_url=api_url,  
        prompt=prompt,  
        history=history,  
        temperature=temperature,  
        max_tokens=max_tokens,  
    )  
)
```

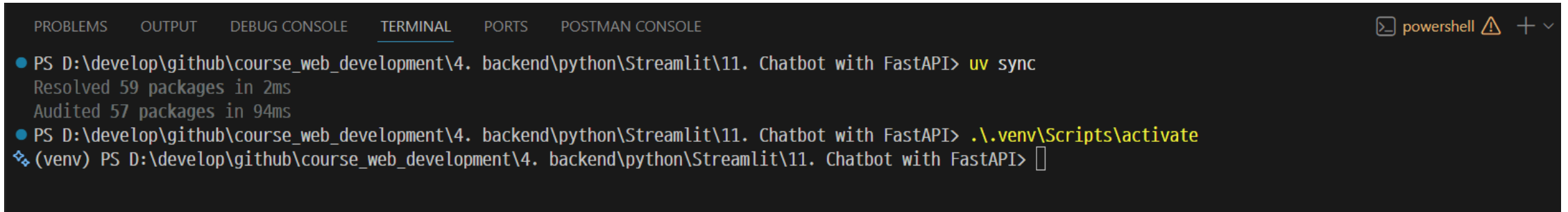
`st.write_stream()` 은 generator가 주는 문자열을 실시간으로 화면에 출력합니다.

서버 실행

가상환경 만들기

가상환경을 만들고 활성화합니다.

```
uv sync
.\.venv\Scripts\activate
```



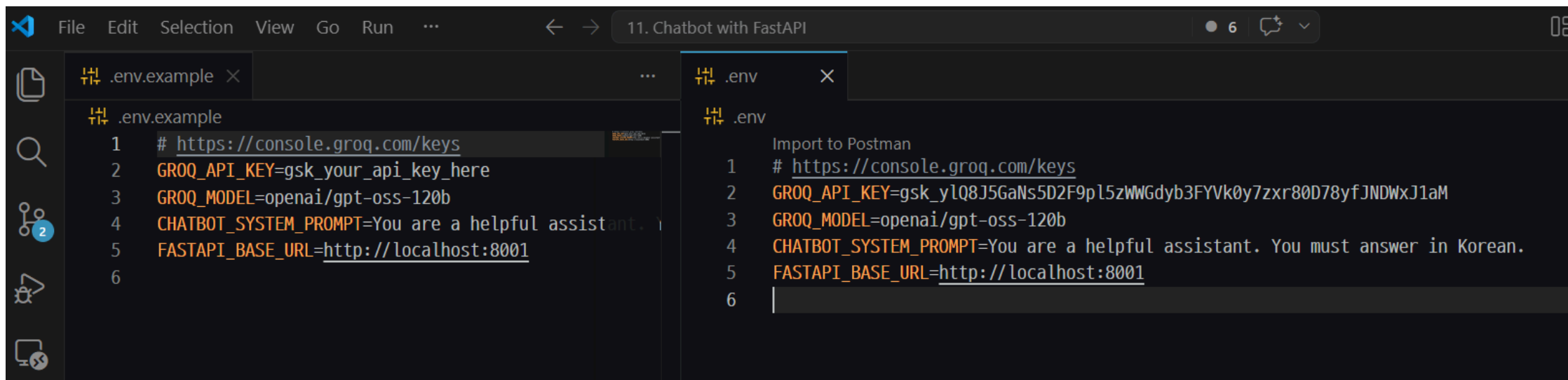
The screenshot shows a VS Code terminal window with the following tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS, and POSTMAN CONSOLE. The terminal is running a PowerShell session. The first command executed is `uv sync`, which resolves 59 packages in 2ms and audits 57 packages in 94ms. The second command is `.\.venv\Scripts\activate`, which successfully activates the virtual environment, indicated by the prompt changing from `PS` to `(venv) PS`.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE powershell ⚠ + v
• PS D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI> uv sync
  Resolved 59 packages in 2ms
  Audited 57 packages in 94ms
• PS D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI> .\.venv\Scripts\activate
(venv) PS D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI> 
```

환경변수

`.env.example` 을 참고해서 `.env` 파일을 만듭니다.

`GROQ_API_KEY` 에는 Groq Console에서 발급한 실제 API Key를 넣습니다.



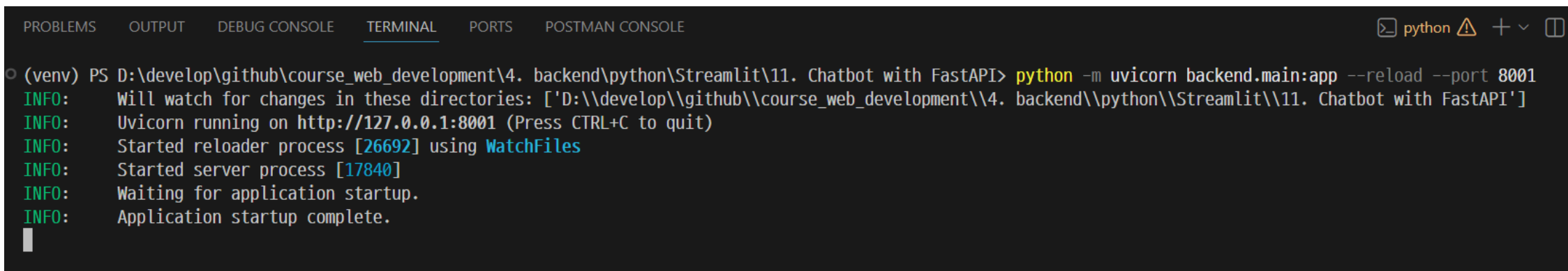
```
File Edit Selection View Go Run ... 11. Chatbot with FastAPI 6
```

```
.env.example
1 # https://console.groq.com/keys
2 GROQ_API_KEY=gsk_your_api_key_here
3 GROQ_MODEL=openai/gpt-oss-120b
4 CHATBOT_SYSTEM_PROMPT=You are a helpful assistant.
5 FASTAPI_BASE_URL=http://localhost:8001
6
```

```
.env
Import to Postman
1 # https://console.groq.com/keys
2 GROQ_API_KEY=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
3 GROQ_MODEL=openai/gpt-oss-120b
4 CHATBOT_SYSTEM_PROMPT=You are a helpful assistant. You must answer in Korean.
5 FASTAPI_BASE_URL=http://localhost:8001
6
```

FastAPI 실행

```
python -m uvicorn backend.main:app --reload --port 8001
```

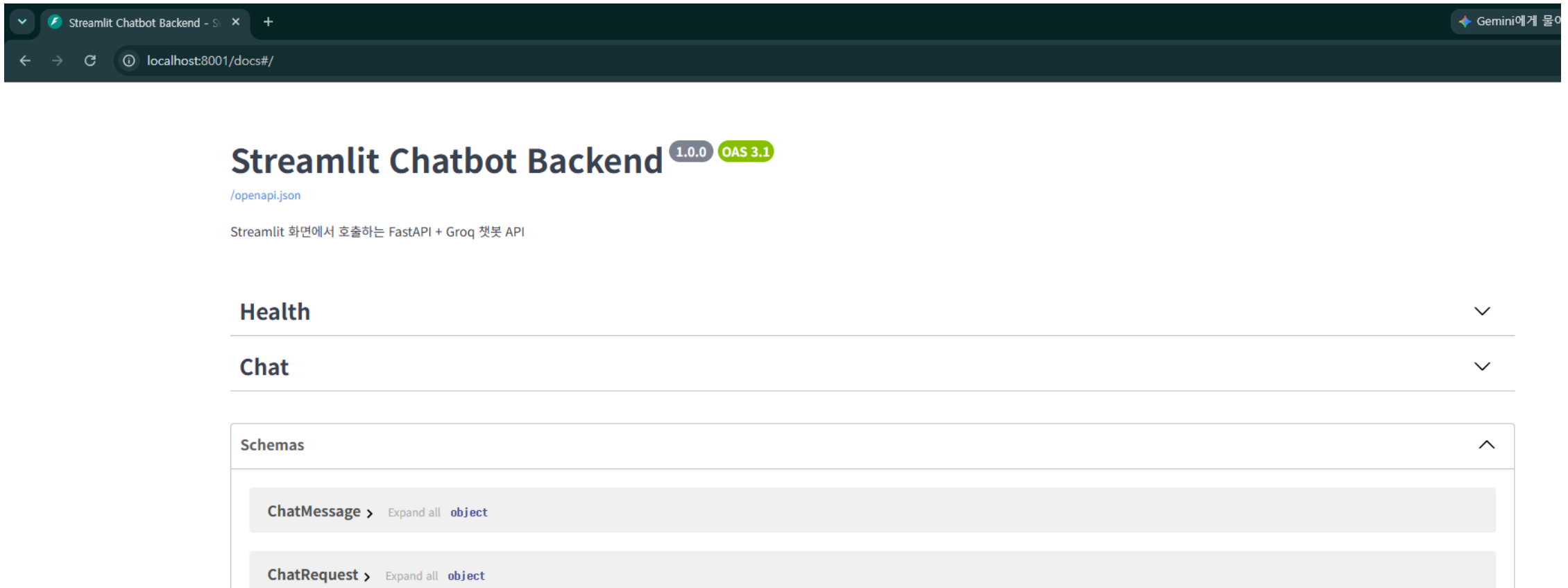


The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), 'PORTS', and 'POSTMAN CONSOLE'. In the top right corner, there is a status bar showing 'python' with a warning icon, a plus sign, a dropdown arrow, and a window icon. The terminal content shows a command prompt prompt '(venv) PS D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI>' followed by the command 'python -m uvicorn backend.main:app --reload --port 8001'. Below the command, there are several lines of output: 'INFO: Will watch for changes in these directories: ['D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI']', 'INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)', 'INFO: Started reloader process [26692] using WatchFiles', 'INFO: Started server process [17840]', 'INFO: Waiting for application startup.', and 'INFO: Application startup complete.' A cursor is visible at the end of the last line of output.

```
(venv) PS D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI> python -m uvicorn backend.main:app --reload --port 8001
INFO: Will watch for changes in these directories: ['D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI']
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO: Started reloader process [26692] using WatchFiles
INFO: Started server process [17840]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Swagger 실습 (<http://localhost:8001/docs>)

각 API에서 Try it out을 누른 뒤 아래 순서대로 입력합니다.



상태 확인

GET /health

GET

/health Health Check

^

Parameters

Cancel

No parameters

Execute

확인할 것:

- 응답 상태 코드가 `200 OK` 입니다.
- `groq_api_key` 가 `configured` 이면 `.env` 에 API Key 형식의 값이 있는 상태입니다.
- `placeholder` 이면 예제값이므로 실제 Groq API Key로 교체합니다.
- `missing` 이면 `.env` 의 `GROQ_API_KEY` 를 확인합니다.
- `/chat` 에서 `Groq API Key가 유효하지 않거나 폐기되었습니다.` 가 나오면 새 API Key를 발급해 `.env` 에 다시 넣습니다.

Code	Details
200	Response body <pre>{ "status": "ok", "groq_api_key": "configured", "model": "openai/gpt-oss-120b" }</pre>  Download Response headers <pre>content-length: 73 content-type: application/json date: Sun, 05 Jul 2026 11:00:52 GMT server: uvicorn</pre>

일반 챗봇 응답

POST /chat

```
{  
  "message": "FastAPI와 Streamlit을 함께 쓰는 이유를 짧게 설명해줘.",  
  "history": [],  
  "temperature": 0.7,  
  "max_tokens": 500  
}
```

POST

/chat 일반 챗봇 응답

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{
  "message": "FastAPI와 Streamlit을 함께 쓰는 이유를 짧게 설명해줘.",
  "history": [],
  "temperature": 0.7,
  "max_tokens": 500
}
```

Execute

확인할 것:

- 응답에 `reply` 가 들어옵니다.
- `elapsed_ms` 로 응답 시간을 확인할 수 있습니다.

Code

Details

200

Response body

```
{
  "reply": "FastAPI와 Streamlit을 함께 쓰면 **백엔드와 프론트엔드**를 각각 최적화된 도구로 빠르게 구축할 수 있습니다.\n\n- **FastAPI**: 고성능 비동기 API 서버를 간단한 코드로 만들고, 자동 문서화(Swagger)와 타입 검증을 제공한다. \n- **Streamlit**: 파이썬 코드만으로 인터랙티브한 웹 UI를 즉시 생성할 수 있어 데이터 시각화·데모 제작에 강점이 있다. \n\n두 개를 결합하면 \n\nFastAPI가 복잡한 비즈니스 로직·데이터베이스·머신러닝 모델을 처리하고, \n\nStreamlit이 사용자가 직접 입력·결과를 확인할 수 있는 UI를 제공한다. \n\n따라서 **빠른 프로토타이핑**과 **생산 수준 API**를 동시에 구현할 수 있어, 데이터 사이언스·AI 프로젝트에서 많이 활용됩니다.",
  "model": "openai/gpt-oss-120b",
  "elapsed_ms": 1107,
  "input_tokens": 104,
  "output_tokens": 287,
  "total_tokens": 391
}
```

 Download

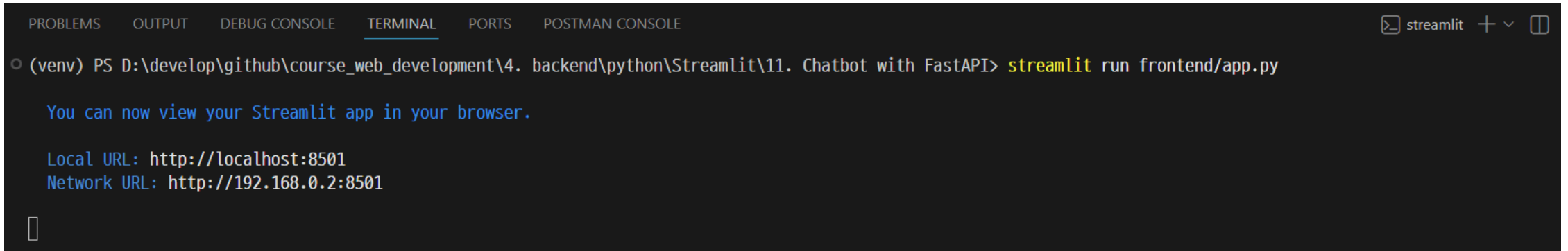
Response headers

```
content-length: 978
content-type: application/json
date: Sun, 05 Jul 2026 11:12:39 GMT
server: uvicorn
```

Streamlit 실행

- FastAPI 실행하고 있는 상태에서 실행

```
streamlit run frontend/app.py
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
streamlit + -

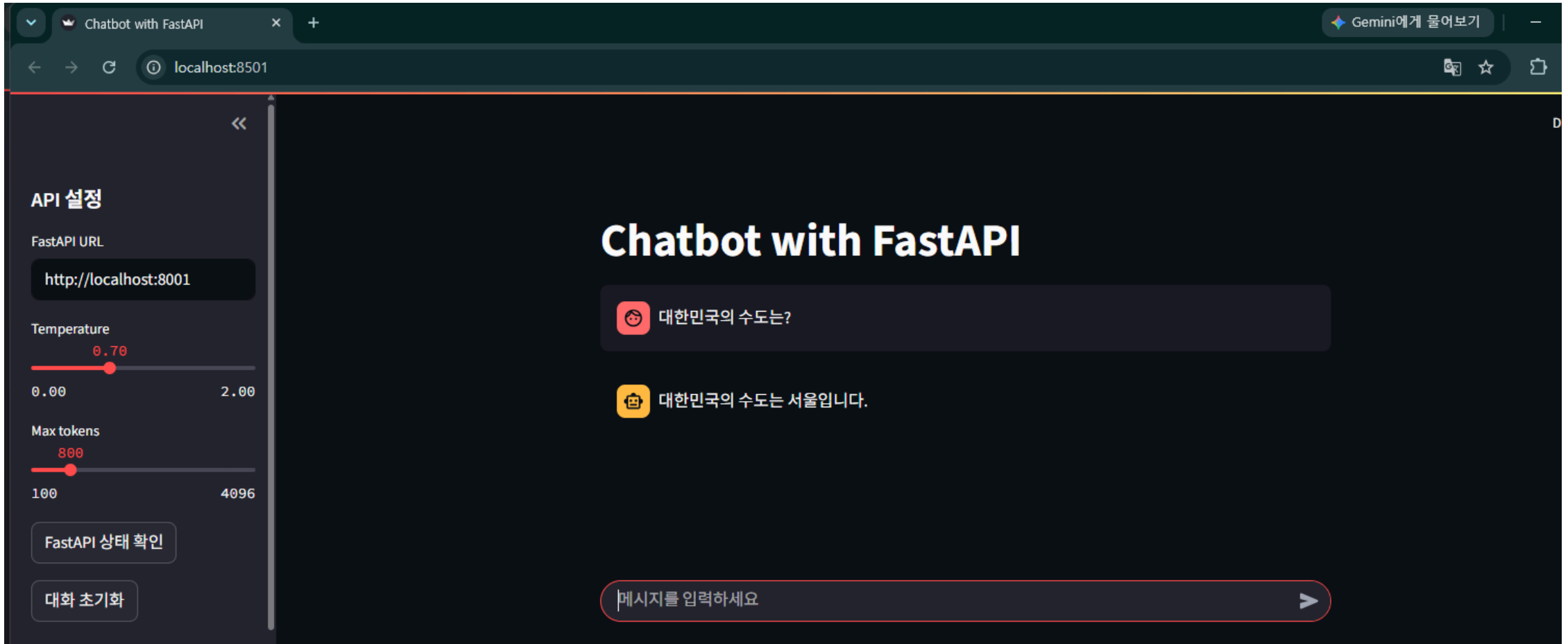
○ (venv) PS D:\develop\github\course_web_development\4. backend\python\Streamlit\11. Chatbot with FastAPI> streamlit run frontend/app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.2:8501

[]
```

챗봇 테스트



FastAPI 상태 확인

API 설정

FastAPI URL

http://localhost:8001

Temperature

0.70

0.00 2.00

Max tokens

800

100 4096

FastAPI 상태 확인 ←

FastAPI 연결 성공

```
{
  "status": "ok"
  "groq_api_key": "configured"
  "model": "openai/gpt-oss-120b"
}
```

대화 초기화

Chatbot with FastAPI

메시지를 입력하세요